
SCALABLE DIFFUSION MODELS WITH TRANSFORMERS

PROYECTO 2 DE MODELOS GENERATIVOS PROFUNDOS

Stefanny Arboleda Ceferino
stefanny.arboleda@cimat.mx

Emmanuel A. Larralde Ortiz
emmanuel.larralde@cimat.mx

ABSTRACT

Diffusion models have emerged as one of the most robust approaches for high-quality image generation, often surpassing GAN- and VAE-based architectures in stability and visual fidelity. However, most successful diffusion models still rely on convolutional U-Nets as the denoising backbone, which limits the direct use of recent advances in Transformer-based architectures. The work *Scalable Diffusion Models with Transformers* (DiT) addresses this by replacing the U-Net with a pure Transformer operating in the latent space of a pretrained VAE, where latents are represented as patch sequences and conditioned on time and class.

In this project, we re-implement in PyTorch the core ideas of DiT in a controlled CIFAR-10 setting. We train a latent diffusion model with $T = 1000$ steps over the latent space of a Stable Diffusion VAE, and implement three Transformer variants proposed in the original paper: (i) *AdaLN-Zero*, where conditioning is injected through adaptive LayerNorm modulations; (ii) *Cross-Attention*, which uses dedicated conditioning tokens; and (iii) *In-Context*, where time and class tokens are concatenated to image tokens. All variants share the same DiT-S/2 backbone and training protocol.

We evaluate sample quality using FID, Inception Score, sFID, and precision/recall, computed with the official OpenAI evaluation code. Our results show that AdaLN-Zero learns more slowly at early epochs but ultimately outperforms the Cross-Attention and In-Context variants on most metrics, while all architectures generate visually coherent CIFAR-10 samples.

Link to our GitHub repository: https://github.com/L4rralde/diffusion_transformers

Keywords Diffusion Transformer · Scalable diffusion · DiT · ViT · VAE

1. Introducción

En los últimos años, los modelos de difusión se han consolidado como una de las familias de modelos generativos más exitosas para la síntesis de imágenes de alta calidad, superando en muchos escenarios a GANs y VAEs tradicionales en términos de estabilidad de entrenamiento y fidelidad visual. Siguiendo trabajos como DDPM y sus extensiones, la idea central consiste en aprender un proceso de denoising que invierte una cadena de difusión progresiva que corrompe los datos con ruido gaussiano; una vez entrenado, el modelo puede generar nuevas muestras partiendo de ruido puro y aplicando de manera iterativa los pasos de denoising condicionados en el tiempo y, opcionalmente, en etiquetas de clase.

La mayoría de los modelos de difusión de referencia para imágenes de alta resolución utilizan arquitecturas convolucionales tipo U-Net como *backbone* del modelo de denoising. Estas arquitecturas explotan inductivas propias de las CNN (equivarianza traslacional, filtros locales, pirámides de resolución), pero presentan ciertas limitaciones para escalar de manera sistemática en tamaño de modelo y resolución, y para reutilizar directamente avances recientes en arquitecturas basadas en Transformers. De forma paralela, la comunidad de visión por computadora ha mostrado que modelos tipo Vision Transformer (ViT) [1] pueden alcanzar un rendimiento competitivo o incluso superior al de las CNN cuando se entrenan a gran escala sobre grandes colecciones de datos.

El trabajo *Scalable Diffusion Models with Transformers* (DiT) [2] propone reemplazar el U-Net estándar por un Transformer puro como red de denoising en un modelo de difusión latente. En lugar de operar directamente en el espacio de píxeles, DiT aplica el proceso de difusión sobre el espacio latente de un autoencoder variacional (VAE) preentrenado, y representa esos latentes como una secuencia de parches (*tokens*) sobre la que actúa un Transformer similar a ViT. Para integrar la información del tiempo de difusión y de la clase, el paper introduce distintos mecanismos de acondicionamiento, entre ellos la variante *AdaLN-Zero* (modulación adaptativa de LayerNorm), y variantes basadas en *cross-attention* e *in-context conditioning*. Los resultados muestran que, con una escala de modelo adecuada, DiT alcanza y supera el rendimiento de arquitecturas convolucionales, manteniendo además una receta de escalamiento clara (más capas, más cabezas, mayor dimensión de embedding).

En este proyecto nos enfocamos en la re-implementación desde cero de las ideas principales de DiT para un escenario controlado de menor escala. Concretamente, entrenamos un modelo de difusión latente sobre CIFAR-10, utilizando como codificador-decodificador un VAE preentrenado de Stable Diffusion, y exploramos de manera sistemática las tres variantes de Transformer (ver Figura 1) propuestas en el artículo original:

- **DiT con AdaLN-Zero:** variante principal en la que el acondicionamiento de tiempo y clase se inyecta mediante modulaciones adaptativas sobre capas de normalización (Adaptive LayerNorm Zero) en cada bloque Transformer, incluyendo la capa de salida.
- **DiT con Cross-Attention:** arquitectura en la que el tiempo y la clase se representan como un conjunto reducido de tokens de condición, que actúan como contexto para capas de *cross-attention* aplicadas sobre los tokens de imagen.
- **DiT con In-Context Conditioning:** enfoque en el que los tokens de tiempo y clase se concatenan directamente a la secuencia de tokens de imagen, de modo que los bloques Transformer estándar operan sobre una secuencia extendida, y el acondicionamiento se incorpora de manera similar al *prompting* en Transformers.

Nuestro objetivo principal no es proponer una nueva arquitectura, sino reconstruir y analizar el diseño de DiT en un entorno reproducible y didáctico. Para ello, implementamos una librería modular en PyTorch que incluye: (i) un *backbone* genérico DiTBase con *patch embeddings* estilo ViT, embeddings de tiempo y clase, y codificación posicional sinusoidal 2D; (ii) las tres variantes de bloques Transformer (AdaLN-Zero, Cross-Attention e In-Context) y sus capas finales correspondientes; y (iii) un flujo de entrenamiento completo en el espacio latente de un VAE preentrenado, acompañado de scripts de muestreo y evaluación mediante FID, Inception Score, sFID y métricas de precision/recall.

Las contribuciones concretas de este proyecto pueden resumirse de la siguiente manera:

- Implementación desde cero, en PyTorch, de un modelo de difusión latente basado en Transformers siguiendo la arquitectura DiT, con un diseño modular que separa claramente el *backbone* y las variantes de acondicionamiento.
- Desarrollo de las tres variantes de Transformer propuestas en el paper (*AdaLN-Zero*, *Cross-Attention* e *In-Context conditioning*), todas entrenadas bajo el mismo protocolo experimental sobre CIFAR-10.
- Integración de una canalización de entrenamiento y evaluación que incluye el uso de un VAE preentrenado de Stable Diffusion, generación de muestras a partir de checkpoints EMA y cálculo sistemático de métricas basadas en Inception para comparar el rendimiento de las distintas variantes.

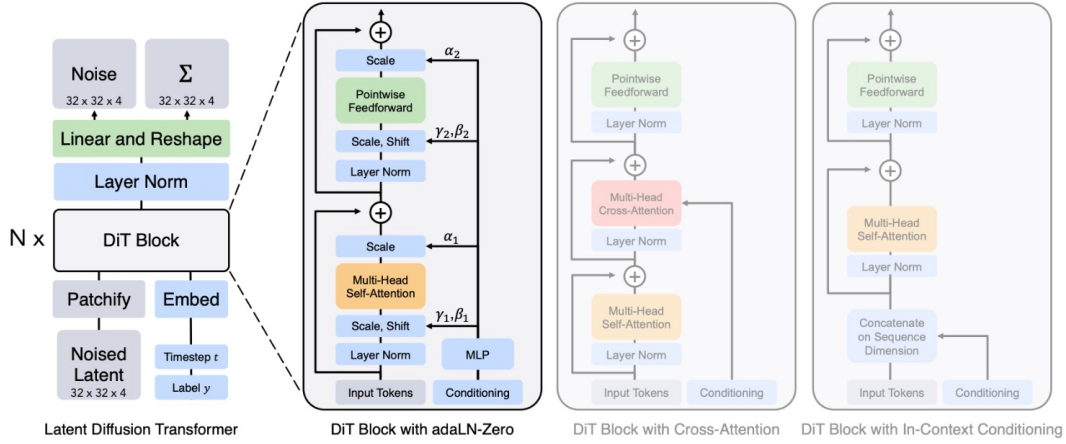


Figura 1: Arquitecturas de Transformer propuestas en el artículo original.

El resto de este documento está organizado de la siguiente manera. En la Sección 2 describimos los detalles de implementación del modelo, el conjunto de datos y el flujo de trabajo de entrenamiento en el espacio latente. La Sección 3 resume las métricas utilizadas para evaluar la calidad y diversidad de las imágenes generadas. En la Sección 4 presentamos los resultados experimentales obtenidos por cada variante de DiT sobre CIFAR-10 y discutimos sus diferencias. Finalmente, en la Sección 5 exponemos las conclusiones principales y posibles líneas de trabajo futuro.

2. Detalles de implementación

En esta sección describimos los aspectos prácticos de la implementación del modelo Diffusion Transformer (DiT) sobre CIFAR-10, incluyendo el preprocesamiento del conjunto de datos, el flujo de trabajo de entrenamiento en el espacio latente de un VAE pre-entrenado y la configuración experimental empleada en las tres variantes de arquitectura: AdaLN-Zero, Cross-Attention e In-Context conditioning.

2.1. Conjunto de datos

Para todos los experimentos utilizamos el conjunto de datos CIFAR-10, que contiene 50 000 imágenes de entrenamiento y 10 000 de prueba, con 10 clases balanceadas (*airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck*). La carga de datos se realiza con la implementación oficial de `torchvision.datasets.CIFAR10`.

En el script `train_dit.py` las imágenes de entrenamiento se preprocesan mediante la siguiente canalización de transformaciones:

- **Redimensionamiento:** todas las imágenes se reescalan a una resolución cuadrada de 64×64 píxeles mediante `transforms.Resize`, lo que permite trabajar posteriormente con latentes de tamaño compatible con la arquitectura de DiT y el VAE (`image_size` múltiplo de 8).
- **Aumento de datos:** se aplica un `RandomHorizontalFlip` con probabilidad $p = 0,5$ para introducir invariancia a volteos horizontales y aumentar la diversidad del conjunto de entrenamiento.

- **Normalización:** las imágenes se convierten a tensores y se normalizan canal a canal con media 0,5 y desviación estándar 0,5, mediante `transforms.Normalize(0.5, 0.5, inplace=True)`, de modo que los valores queden aproximadamente en el rango $[-1, 1]$.

Estas transformaciones se encapsulan en el método `get_cifar10_data_loader`, que construye un `DataLoader` con `batch_size` predeterminado de 128, barajado aleatoriamente (`shuffle=True`), `num_workers=4`, `pin_memory=True` y `drop_last=True`.

Para la evaluación basada en FID se prepara un conjunto de referencia coherente con la resolución de muestreo. El script `cifar10_to_folder.py` genera una versión reescalada y recortada de las imágenes de prueba de CIFAR-10 a 64×64 mediante una rutina de *center crop* adaptada de ADM, y guarda cada imagen en disco con el nombre `sample_{id}_{clase}.png`. A partir de estas imágenes se construye posteriormente un archivo `.npz` de referencia usando `samples_to_npz.py`, que apila las muestras en un tensor $(N, H, W, 3)$ en formato `uint8`.

2.2. Flujo de Trabajo y Metodología de Entrenamiento

El entrenamiento de las variantes de DiT se realiza en el **espacio latente** de un autoencoder variacional pre-entrenado, siguiendo la filosofía de los modelos de difusión latente.

Codificación al espacio latente. Se utiliza el modelo `AutoencoderKL` de `diffusers`, cargado desde `stabilityai/sd-vae-ft-{mse, ema}`, donde en nuestros experimentos empleamos por defecto la variante `ema`. El VAE se mueve al dispositivo (`cuda` o `cpu`), se fija en modo evaluación (`eval()`) y todos sus parámetros se congelan (`requires_grad=False`) para que no participe en el entrenamiento.

Dada una imagen preprocesada $x \in \mathbb{R}^{3 \times 64 \times 64}$, el VAE produce una distribución posterior $q(z | x)$ y se muestrea un latente

$$z \sim q(z | x), \quad z \in \mathbb{R}^{4 \times 8 \times 8},$$

que posteriormente se reescala por el factor constante utilizado en Stable Diffusion,

$$\tilde{z} = 0,18215 \cdot z,$$

para mantener la magnitud de los latentes en un rango adecuado para el modelo de difusión.

Proceso de difusión directo. Sobre los latentes $\tilde{z}$ se define un proceso de difusión discreto de T pasos mediante un *schedule* lineal de varianzas $\{\beta_t\}_{t=1}^T$, implementado en la clase `DiffusionSchedule` (`diffusion.py`). En nuestros experimentos usamos $T = 1000$, con

$$\beta_t \sim \text{linspace}(10^{-4}, 2 \times 10^{-2}), \quad \alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

Para cada latente de entrenamiento se muestrea un tiempo $t \sim \mathcal{U}\{0, \dots, T-1\}$ y ruido gaussiano $\varepsilon \sim \mathcal{N}(0, I)$. El estado ruidoso se genera con

$$x_t = \sqrt{\bar{\alpha}_t} \tilde{z} + \sqrt{1 - \bar{\alpha}_t} \varepsilon,$$

implementado en el método `q_sample`.

Modelo de predicción de ruido. El modelo DiT recibe como entrada el latente ruidoso x_t , el *time-step* discreto t (embebido mediante `TimestepEmbedder`) y la clase de CIFAR-10 y (embebida mediante `LabelEmbedder`). Las tres variantes de arquitectura comparten el mismo backbone `DiTBase` (`dit_core.py`), que:

- Divide el latente x_t en parches mediante `PatchEmbed`, generando una secuencia de tokens de dimensión D similar a un ViT.

- Suma embeddings posicionales sinusoidales 2D fijos sobre la grilla de parches.
- Combina las embeddings de tiempo y clase para incorporar conditioning.

En entrenamiento, el modelo se optimiza para predecir el ruido añadido:

$$\varepsilon_{\theta}(x_t, t, y) \approx \varepsilon,$$

minimizando una pérdida MSE

$$\mathcal{L} = \mathbb{E}_{t, \varepsilon} [\|\varepsilon - \varepsilon_{\theta}(x_t, t, y)\|_2^2].$$

Entrenamiento principal. El bucle de entrenamiento implementado en `train_dit.py` para cada epoch y batch realiza:

1. Carga de un lote de imágenes y etiquetas de CIFAR-10 mediante `DataLoader`.
2. Codificación de las imágenes al espacio latente con el VAE pre-entrenado y reescalado por el factor 0.18215 (sin gradiente).
3. Muestreo de timesteps t y ruido gaussiano ε .
4. Construcción de x_t a través de `schedule.q_sample`.
5. Evaluación del modelo DiT para obtener $\varepsilon_{\theta}(x_t, t, y)$.
6. Cálculo de la pérdida MSE, retropropagación, *gradient clipping* y actualización de los pesos mediante AdamW.
7. Actualización de los parámetros suavizados mediante un esquema de *Exponential Moving Average* (EMA).

Cada 10 epochs se guardan checkpoints del modelo base y de su versión EMA (archivos `*.pt`) en el directorio de resultados, junto con estadísticas de entrenamiento (loss vs. iteración) en un archivo `.npz`. La versión EMA de los pesos se emplea posteriormente para la generación de muestras, ya que suele producir una calidad de imagen más estable.

Muestreo y reverse process. El proceso de muestreo se apoya en el paso de reverse diffusion implementado en `p_sample_step`. Dado un estado x_t , una etiqueta y y un timestep t , se combina la predicción de ruido condicional y no condicional mediante *classifier-free guidance* (CFG):

- Se evalúa el modelo de difusión con la etiqueta real y y con un token nulo adicional (clase índice 10 para CIFAR-10), obteniendo $\varepsilon_{\text{cond}}$ y $\varepsilon_{\text{uncond}}$.
- Se construye una predicción guiada

$$\varepsilon_{\theta}^{\text{CFG}} = \varepsilon_{\text{uncond}} + s(\varepsilon_{\text{cond}} - \varepsilon_{\text{uncond}}),$$

donde s es el factor de guidance (`cfg_scale`).

- A partir de $\varepsilon_{\theta}^{\text{CFG}}$ se aplica la ecuación de DDPM para muestrear x_{t-1} con media μ_t y varianza β_t .

Tras iterar desde $t = T - 1$ hasta $t = 0$, se obtiene un latente limpio que se decodifica con el VAE para recuperar una imagen RGB en el espacio de píxeles. El script `sample_all_checkpoints.sh` automatiza este proceso para todos los checkpoints EMA de un tipo de modelo dado, generando directorios de muestras por época. Posteriormente, `samples_to_npz.py` convierte las imágenes generadas en un archivo `samples.npz`, que se utiliza en el cálculo de FID, Inception Score, sFID y métricas de precisión/recobrado mediante `evaluator.py` (basado en Inception v3 en TensorFlow).

2.3. Configuración Experimental

2.3.1. Parámetros de la Arquitectura

A nivel de código, el modelo de denoising se organiza en dos niveles: (i) un *backbone* genérico definido en `dit_core.py`, que se encarga de representar los latentes como secuencias tipo ViT y de incorporar los embeddings de tiempo y clase; y (ii) tres variantes de bloques Transformer, cada una en un archivo independiente (`dit_adaln_zero.py`, `dit_cross_attention.py`, `dit_in_context.py`), que difieren únicamente en la forma en que inyectan la condición en el modelo.

Backbone genérico (`dit_core.py`). La clase `DiTBase` implementa la estructura común para las tres variantes:

- **Dimensión Latente (z_{dim}):** el VAE codifica las imágenes de 64×64 a un mapa latente de tamaño $\mathbb{R}^{4 \times 8 \times 8}$, es decir, cuatro canales con resolución espacial 8×8 .
- **Parches y número de tokens:** el módulo `PatchEmbed` aplica una convolución 2D con kernel y stride $= 2$ sobre el latente, dividiéndolo en parches 2×2 . Esto produce una grilla 4×4 de parches y una secuencia de tokens de longitud $N = 16$, cada uno con dimensión de embedding D .
- **Dimensión del embedding:** el tamaño de embedding se fija en $D = 512$ (`hidden_size`), tanto para los tokens de imagen como para los embeddings de tiempo y clase.
- **Embeddings de tiempo:** la clase `TimestepEmbedder` primero construye un embedding sinusoidal de dimensión 256 a partir del timestep escalar t , y luego lo proyecta a dimensión D mediante un MLP de dos capas con activación SiLU.
- **Embeddings de clase:** la clase `LabelEmbedder` define una tabla de embeddings de tamaño $(\text{num_classes} + 1, D)$, donde el índice adicional se usa como token nulo para *classifier-free guidance*. Durante el entrenamiento puede aplicar *token dropout* para simular condicionamiento no condicional.
- **Positional encodings:** se construyen embeddings posicionales 2D sinusoidales fijos (`get_2d_sincos_pos_embed`), que se suman a la secuencia de tokens de imagen. Estos embeddings se almacenan como un parámetro no entrenable `pos_embed`.
- **Bloques y capa final:** `DiTBase` declara una lista de bloques Transformer (`self.blocks`) y una capa final (`self.final_layer`) que se especializan en cada variante. La función `unpatchify` toma los tokens de salida y los reensambla en un mapa latente de tamaño $\mathbb{R}^{C_{\text{out}} \times 8 \times 8}$.
- **Profundidad y atención:** en todos los experimentos se utilizan $L = 8$ bloques (`depth=8`) con `num_heads = 8` cabezas de atención multi-cabeza. Las capas lineales se inicializan con Xavier y los embeddings posicionales se fijan al inicio.

Variante AdaLN-Zero (`dit_adaln_zero.py`). La clase `DiT_AdaLNZero` implementa la variante principal del paper, en la cual el acondicionamiento de tiempo y clase se inyecta mediante modulaciones adaptativas de LayerNorm (AdaLN-Zero):

- Para cada latente ruidoso x_t , se obtienen un embedding de tiempo t_{emb} y uno de clase y_{emb} ; su suma define una condición global $c = t_{\text{emb}} + y_{\text{emb}} \in \mathbb{R}^D$.
- Cada bloque `AdaLNZeroBlock` contiene:
 - Una rama de self-attention (`Attention`) aplicada sobre los tokens de imagen.
 - Una MLP de dos capas con activación GELU.
 - Una red `adaLN_modulation` que, a partir de c , produce seis vectores: $\text{shift}_{\text{msa}}, \text{scale}_{\text{msa}}, \text{gate}_{\text{msa}}, \text{shift}_{\text{mlp}}, \text{scale}_{\text{mlp}}, \text{gate}_{\text{mlp}} \in \mathbb{R}^D$.

- La normalización `LayerNorm` se aplica sin parámetros aprendibles, y luego se modula con la función `modulate`:

$$\text{modulate}(x, \text{shift}, \text{scale}) = x \cdot (1 + \text{scale}) + \text{shift}.$$

- Las salidas de atención y MLP se combinan con residuales controlados por compuertas:

$$\begin{aligned} x &\leftarrow x + \text{gate}_{\text{msa}} \cdot \text{MSA}(\text{modulate}(\text{LN}(x), \text{shift}_{\text{msa}}, \text{scale}_{\text{msa}})), \\ x &\leftarrow x + \text{gate}_{\text{mlp}} \cdot \text{MLP}(\text{modulate}(\text{LN}(x), \text{shift}_{\text{mlp}}, \text{scale}_{\text{mlp}})). \end{aligned}$$

- La capa final `AdaLNZeroFinalLayer` aplica otra modulación `AdaLN` sobre los tokens de salida y una proyección lineal que mapea cada token al vector correspondiente a un parche del latente de salida (con canales de media y varianza si `learn_sigma=True`).
- Todos los pesos de la última capa de `AdaLN` en cada bloque (y de la capa final) se inicializan a cero, lo que garantiza que el modelo comience cerca de una identidad residual y vaya “activando” las modulaciones a medida que aprende.

Variante Cross-Attention (`dit_cross_attention.py`). En `DiT_CrossAttention`, el acondicionamiento se incorpora mediante capas explícitas de *cross-attention*:

- A partir de los embeddings t_{emb} y y_{emb} se construyen dos *tokens de condición* que se apilan en un tensor de forma $(B, 2, D)$, denominado `cond_tokens`.
- Cada `CrossAttentionBlock` aplica:
 - Self-attention estándar (`Attention`) sobre los tokens de imagen, con residual: $x \leftarrow x + \text{MSA}(\text{LN}(x))$.
 - Una capa de *cross-attention* (`CrossAttention`), donde las queries son los tokens de imagen y las keys/values provienen de `cond_tokens`: $x \leftarrow x + \text{CrossAttn}(\text{LN}(x), \text{cond_tokens})$.
 - Una MLP residual estándar: $x \leftarrow x + \text{MLP}(\text{LN}(x))$.
- La capa final `CrossAttentionFinalLayer` consiste en una `LayerNorm` seguida de una proyección lineal hacia el espacio de parches del latente de salida. En esta variante, el acondicionamiento ya ha sido absorbido por las operaciones de *cross-attention* en los bloques.

Variante In-Context (`dit_in_context.py`). Finalmente, `DiT_InContext` utiliza un esquema de acondicionamiento tipo *prompting*, añadiendo los tokens de tiempo y clase directamente a la secuencia:

- Se construyen los tokens de imagen $x_{\text{tokens}} \in \mathbb{R}^{B \times N \times D}$ mediante `PatchEmbed + pos_embed`, y los dos tokens de condición (t_{emb} y y_{emb}) se apilan en `cond_tokens` $(B, 2, D)$.
- La secuencia completa se forma como:

$$\text{seq} = [x_{\text{tokens}}, \text{cond_tokens}] \in \mathbb{R}^{B \times (N+2) \times D}.$$

- Cada `InContextBlock` es un bloque Transformer estándar:

$$\text{seq} \leftarrow \text{seq} + \text{MSA}(\text{LN}(\text{seq})), \quad \text{seq} \leftarrow \text{seq} + \text{MLP}(\text{LN}(\text{seq})),$$

de modo que la información de los tokens de condición puede fluir hacia los tokens de imagen a través de la atención global.

- Al final, sólo se conservan los primeros N tokens (correspondientes a los parches de imagen) y se descartan los dos tokens de condición. Estos N tokens se pasan por `InContextFinalLayer` (`LayerNorm + Linear`) y, tras `unpatchify`, se obtiene el latente de salida.

En resumen, todas las variantes comparten la misma representación latente, el mismo esquema de *patch embedding* y el mismo tamaño de modelo; la diferencia radica exclusivamente en cómo se introduce la información de tiempo y clase dentro del Transformer, lo que nos permite comparar de forma controlada el impacto de `AdaLN-Zero`, `Cross-Attention` e `In-Context conditioning` sobre la capacidad del modelo de difusión.

2.3.2. Parámetros de Entrenamiento

Entrenamiento General:

- **Optimizador:** AdamW aplicado a todos los parámetros del modelo DiT, con tasa de aprendizaje inicial $\text{lr} = 1 \times 10^{-4}$, $\text{betas} = (0,9, 0,999)$ y $\text{weight_decay} = 0$.
- **Tamaño de lote:** $\text{batch_size} = 128$ imágenes por iteración.
- **Número de epochs:** el modelo se entrena durante 100 epochs sobre el conjunto de entrenamiento de CIFAR-10.
- **Pasos de difusión:** se fija el número de pasos de difusión a $T = 1000$ ($\text{num_diffusion_steps}$), con $\beta_{\text{start}} = 10^{-4}$ y $\beta_{\text{end}} = 2 \times 10^{-2}$.
- **Gradient clipping:** antes de la actualización del optimizador se aplica recorte de gradiente global con norma L2 máxima $\text{grad_clip} = 1,0$ para mejorar la estabilidad numérica del entrenamiento.
- **EMA de parámetros:** se mantiene una copia suavizada de los pesos del modelo mediante un esquema de Exponential Moving Average con factor de decaimiento $\text{ema_decay} = 0,9999$. Esta versión EMA es la que se utiliza para la generación de muestras y evaluación cuantitativa.
- **Dispositivo y precisión:** todos los experimentos se ejecutan en GPU ($\text{device}='cuda'$) cuando está disponible, activando soporte para TF32 en operaciones de multiplicación de matrices ($\text{torch.backends.cuda.matmul.allow_tf32}=\text{True}$ y $\text{torch.backends.cudnn.allow_tf32}=\text{True}$) para acelerar el cómputo manteniendo una calidad numérica adecuada.

En conjunto, esta configuración reproduce fielmente la filosofía de *Scalable Diffusion Models with Transformers* en el contexto de CIFAR-10: el modelo se entrena en el espacio latente de un VAE pre-entrenado, con conditioning en clase y tiempo, y se evalúa generando muestras a partir de la versión EMA de DiT y comparándolas con las imágenes reales mediante métricas basadas en Inception, como se detalla en la siguiente sección.

3. Métricas de Evaluación

Para evaluar cuantitativamente la calidad de las muestras generadas por las distintas variantes de Diffusion Transformer sobre CIFAR-10, utilizamos un conjunto de métricas estándar basadas en las activaciones de la red Inception v3: la Distancia de Incepción de Fréchet (FID), el Inception Score (IS), una variante espacial de FID (sFID) y métricas de precisión/recobrado en el espacio de características. Todas estas métricas se calculan utilizando **sin modificaciones** el código de evaluación publicado por OpenAI para modelos de difusión¹, que en nuestro proyecto se agrupa en el script `evaluator.py`, el cual carga un grafo preentrenado de Inception v3 en TensorFlow y opera sobre archivos `.npz` que contienen lotes de imágenes en formato `uint8`.

3.1. Distancia de Incepción de Fréchet (FID —Frechet Inception Distance—)

Descripción Conceptual. La métrica FID mide la distancia entre la distribución de características de las imágenes reales y la de las imágenes generadas, ambas extraídas de una red de clasificación profunda preentrenada (Inception v3).

Sea p_r la distribución de imágenes reales y p_g la distribución del modelo generativo. Al pasar estas imágenes por Inception, se obtienen vectores de características de dimensión d (en nuestro caso, las activaciones de la capa `pool_3`). Suponiendo que estas características se distribuyen aproximadamente como gaussianas multivariadas:

$$\phi(x_r) \sim \mathcal{N}(\mu_r, \Sigma_r), \quad \phi(x_g) \sim \mathcal{N}(\mu_g, \Sigma_g),$$

¹Tomado del repositorio público de OpenAI, adaptado únicamente en rutas y parámetros de entrada.

la Distancia de Incepción de Fréchet se define como:

$$\text{FID}(p_r, p_g) = \|\mu_r - \mu_g\|_2^2 + \text{Tr} \left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2} \right).$$

Valores de FID más bajos indican que las distribuciones de características de las muestras generadas y reales son más similares, es decir, que el modelo produce imágenes más realistas y cercanas al conjunto de referencia.

Implementación. En el código de evaluación que utilizamos (usado directamente del repositorio oficial de OpenAI), las características se extraen con una instancia de la clase `Evaluator` en `evaluator.py`. Este script:

- Descarga, si es necesario, el grafo preentrenado de Inception v3 desde un repositorio público (`classify_image_graph_def.pb`) y lo carga en un grafo de TensorFlow.
- Define dos salidas principales: las activaciones de la capa `pool_3` (`FID_POOL_NAME`) y de una capa convolucional intermedia (`FID_SPATIAL_NAME`), que se utilizan para FID y sFID respectivamente.
- Procesa los archivos `.npz` de referencia y de muestras generadas leyendo por lotes la matriz `arr_0` (tensor de forma $(N, H, W, 3)$) y pasando estos lotes por Inception para obtener las activaciones.

Las estadísticas μ y Σ se encapsulan en la clase `FIDStatistics`, que implementa el método `frechet_distance` para calcular FID entre dos conjuntos de estadísticas. Para cada experimento:

1. Se construye un archivo `ref_cifar10.npz` a partir de las imágenes reales de CIFAR-10 (prueba), usando la canalización `cifar10_to_folder.py + samples_to_npz.py`.
2. Se generan imágenes con el modelo DiT (usando los checkpoints EMA) y se guardan en un directorio específico por época (`sample_all_checkpoints.sh`), que luego se convierte a `samples.npz` con `samples_to_npz.py`.
3. El script `evaluator.py` se ejecuta con:

```
python evaluations/evaluator.py ref_cifar10.npz
                                outputs/{modelo}/samples.npz
```

reportando en consola el valor de FID, junto con otras métricas descritas a continuación.

De esta manera, todos los modelos se comparan bajo las mismas condiciones, utilizando el mismo conjunto de referencia y la misma arquitectura de Inception para extraer las características.

3.2. Inception Score (IS)

Descripción Conceptual. El Inception Score evalúa la calidad y diversidad de un conjunto de imágenes generadas a partir de las distribuciones de probabilidad de clase producidas por una red de clasificación preentrenada. Dada una imagen generada x , la red produce una distribución $p(y | x)$ sobre las clases; el IS se define como:

$$\exp \left(\mathbb{E}_{x \sim p_g} \left[\text{KL}(p(y | x) \| p(y)) \right] \right),$$

donde $p(y) = \mathbb{E}_{x \sim p_g} [p(y | x)]$ es la distribución marginal de clases sobre todas las imágenes generadas.

Intuitivamente, un IS alto se obtiene cuando:

- Para cada imagen, $p(y | x)$ es de baja entropía (la red está muy segura de una clase concreta), lo que indica que las imágenes son nítidas y reconocibles.
- La distribución marginal $p(y)$ tiene alta entropía (las clases están balanceadas), indicando diversidad en las muestras.

Implementación. En `evaluator.py`, siguiendo exactamente la implementación de OpenAI, el Inception Score se calcula a partir de las activaciones de Inception como sigue:

- Se reutiliza el grafo de Inception v3, obteniendo las activaciones de la capa previa a softmax (logits) mediante la variable "softmax/logits/MatMul".
- A partir de estos logits se construye un grafo de softmax que produce distribuciones (N, C) sobre las clases.
- En la función `compute_inception_score`, estas distribuciones se subdividen en bloques de tamaño fijo y se calcula la divergencia KL entre $p(y | x)$ y la distribución marginal $p(y)$ para cada bloque, siguiendo la implementación clásica de Improved GANs.
- Finalmente, se promedian los valores de IS sobre todos los bloques para obtener un único valor escalar.

El IS se reporta principalmente como una referencia adicional de calidad/diversidad, complementando a FID, que suele ser más robusto cuando se compara con un conjunto de referencia concreto.

3.3. FID Espacial (sFID) y Métricas de Precisión/Recall

sFID: Distancia de Fréchet Espacial. Además del FID estándar sobre las activaciones `pool_3`, se calcula una variante espacial (sFID) utilizando activaciones intermedias de Inception v3 (`FID_SPATIAL_NAME = "mixed_6/conv:0"`). Estas activaciones conservan cierta estructura espacial y se truncan a una resolución fija, lo que permite capturar mejor artefactos locales y detalles de textura.

El procedimiento es análogo al de FID:

- Se extraen las activaciones espaciales para el conjunto de referencia y el conjunto generado.
- Se vectorizan y se calculan estadísticas de media y covarianza para cada conjunto.
- Se aplica la misma fórmula de Fréchet entre ambas distribuciones de características espaciales.

El valor resultante (sFID) se reporta junto con FID, proporcionando una medida complementaria que es más sensible a inconsistencias locales en las imágenes generadas.

Precisión y Recall en el espacio de características. Finalmente, `evaluator.py` calcula métricas de *precisión* y *recall* en el espacio de activaciones de Inception, siguiendo el enfoque de *Improved Precision and Recall for Generative Models*. La idea es aproximar la variedad (manifold) de las características de las imágenes reales y generadas, y medir:

- **Precisión:** fracción de muestras generadas cuyas características caen dentro de la vecindad (hiper-esfera) de al menos una muestra real. Mide cuán realistas son las muestras (sin contar diversidad).
- **Recall:** fracción de muestras reales cubiertas por la variedad de las muestras generadas. Mide hasta qué punto el modelo logra cubrir la diversidad del conjunto real.

En el código:

- La clase `ManifoldEstimator` estima radios de vecindad (`manifold_radii`) para cada muestra a partir de distancias k -NN en el espacio de características.
- A partir de estos radios, la función `evaluate_pr` determina si cada punto de un conjunto cae dentro de la vecindad de algún punto del otro conjunto, devolviendo vectores de precisión y recall que se promedian para obtener valores escalares.

Tabla 1: Comparación cuantitativa de modelos con métricas estándar de evaluación de calidad de generación.

Model	Inception Score $\uparrow$	FID $\downarrow$	sFID $\downarrow$	Precision $\uparrow$	Recall $\uparrow$
adaln	7.669	83.27	123.75	0.411	0.374
cross	6.999	85.67	125.26	0.408	0.393
incontext	7.133	87.75	125.67	0.402	0.392

Estas métricas permiten descomponer el desempeño del modelo en términos de *calidad* (precisión) y *cobertura* (recall) del soporte de la distribución real, proporcionando una imagen más rica del comportamiento de cada variante de Diffusion Transformer más allá de FID e IS.

Estas implementaciones se toman directamente del código de evaluación publicado por OpenAI; nuestro aporte se limita a generar los conjuntos de referencia y de muestras en formato `.npz` y a invocar dichas funciones con nuestros datos.

4. Resultados

Adicionalmente, se enfatiza que los resultados a mostrarse fueron ejecutados en una GPU NVIDIA GeForce RTX 5060 Ti, en un entorno de PyTorch con soporte para CUDA; y se pueden encontrar los códigos utilizados para cada etapa en de nuestro repositorio de GitHub.

Se entrenaron modelos de las tres arquitecturas implementadas: *AdaLN-Zero* (adaln), *Cross-Attention* (cross), *In-Context conditioning* (incontext) por 100 épocas utilizando la configuración de transformer *DiT-S/2* (mismas dimensiones que *ViT-S*, pero con un tamaño de parche de 2×2). Para cada arquitectura se almacenan los pesos del modelo entrenado y de un modelo que se actualiza con *Exponential Moving Average* (EMA). Todos los resultados mostrados provienen de utilizar los modelos cuyos pesos son actualizados utilizando el método EMA.

En la figura 2 se observa que el modelo *AdaLN-Zero* tiene un proceso de aprendizaje más lento, alcanzando niveles similares a todos los demás en todas las métricas después de 50 épocas de entrenamiento. En general, todos los modelos alcanzan valores similares en todas las métricas al finalizar el entrenamiento; sin embargo, destaca que *AdaLN-Zero* supera en *Inception* a los otros dos modelos sin mostrar tendencias de estancamiento. De hecho, observamos en la tabla 1 que el modelo con arquitectura *AdaLN-Zero* supera a los otros dos en 4 de las 5 métricas utilizadas. No obstante, el *ranking* podría cambiar al utilizar más épocas de entrenamiento, ya que el aprendizaje de *AdaLN-Zero* podría no haberse estancado aún.

Estos resultados coinciden con los de [2], donde *AdaLN-Zero* generalmente obtiene los mejores resultados en métricas estándar de generación para todos los tamaños de *Transformer*.

En cuanto a la comparación cualitativa en la tarea de generación de muestras, la figura 3 incluye la generación condicionada de los tres modelos para siete clases diferentes de CIFAR-10. Se consigue que los modelos partan de la misma condición y latente z_T usando la misma semilla manual. En esta figura se puede ver que todos los modelos son capaces de generar imágenes consistentes con las condiciones de entrenamiento.

Por último, en la figura 4 se expone la evolución de la generación de muestras por época de entrenamiento del modelo *AdaLN-Zero*. Durante las primeras 30 épocas, la generación no es densa, pues hay varios huecos. Es hasta la época 60 donde estos patrones ‘nebulosos’ y con huecos desaparecen. También se destaca que la magnitud del cambio de las muestras generadas es más notable en algunas categorías que en otras, indicando indicios de que existen clases para las cuales es más fácil aprender el proceso inverso de $z_T \rightarrow z_0$. En general, las clases de *ship*, *automobile*, *airplane* y *horse* son más fáciles de ‘aprender’.

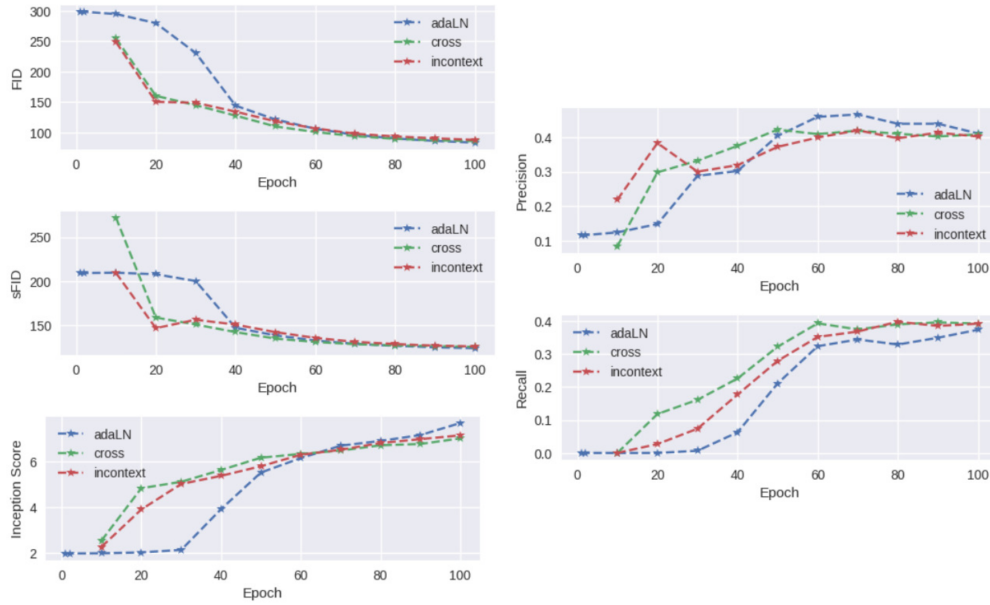


Figura 2: Evaluación cuantitativa de los modelos a lo largo del entrenamiento usando las métricas estándares para generación de imágenes. Todos los modelos tienen *checkpoints* en las épocas 10, 20, ..., 100; excepto *AdaLN-Zero* que además incluye *checkpoints* en las épocas 1 y 2

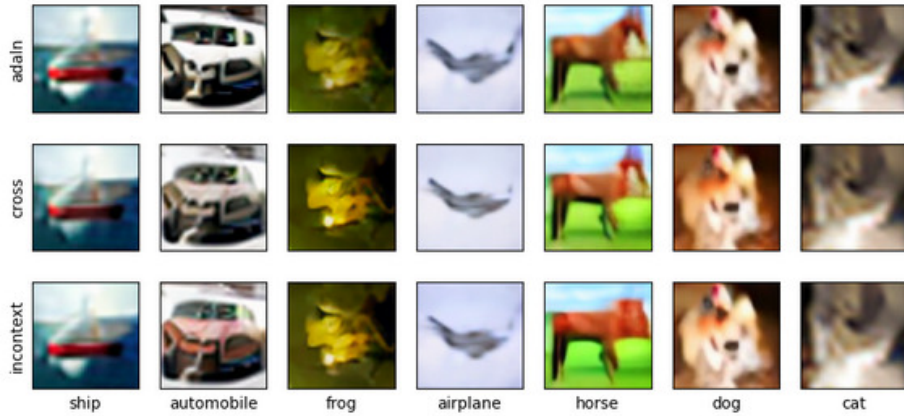


Figura 3: Evaluación cualitativa de los tres modelos entrenados por 100 épocas en siete de las diez clases de CIFAR-10. Se utilizó una semilla manual.

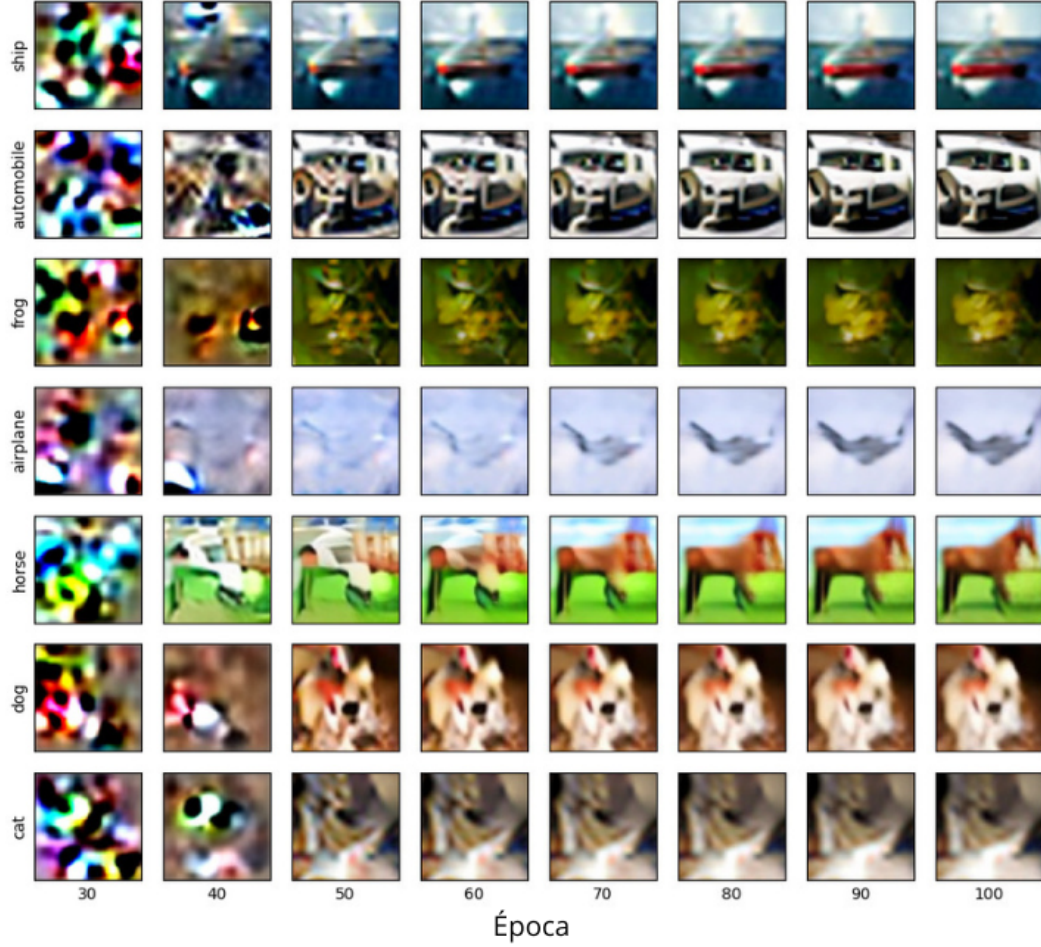


Figura 4: Imágenes generadas usando *checkpoints* de diferentes épocas de entrenamiento del modelo *AdaLN-Zero*. Se muestran 7 categorías diferentes de las 10 de CIFAR-10.

5. Conclusiones

En este trabajo re-implementamos de forma modular el enfoque *Diffusion Transformer* (DiT) propuesto en *Scalable Diffusion Models with Transformers*, adaptado a un escenario de menor escala sobre CIFAR-10 y al uso de un VAE preentrenado de Stable Diffusion. A partir de un *backbone* común tipo ViT (*DiT-S/2*) en el espacio latente, construimos y entrenamos tres variantes de acondicionamiento en tiempo y clase: AdaLN-Zero, Cross-Attention e In-Context conditioning, manteniendo constante la configuración de entrenamiento y el protocolo de evaluación.

Desde el punto de vista cuantitativo, los resultados muestran que las tres variantes alcanzan desempeños comparables en FID, sFID, precisión y recall tras 100 épocas de entrenamiento, evidenciando que todas son capaces de modelar de manera razonable la distribución de CIFAR-10 en el espacio latente. No obstante, la arquitectura basada en AdaLN-Zero destaca sistemáticamente: aunque su convergencia es más lenta en las primeras épocas, termina obteniendo los mejores resultados en 4 de las 5 métricas evaluadas (incluyendo Inception Score y FID), y no presenta signos claros de estancamiento, lo que sugiere que podría beneficiarse

de entrenamientos más prolongados. Las variantes de Cross-Attention e In-Context, por su parte, muestran un comportamiento competitivo y estable, con ligeras ventajas relativas en recall, lo que indica una mejor cobertura de la diversidad del conjunto real en algunos casos.

Cualitativamente, las muestras generadas por los tres modelos son consistentes con las etiquetas de clase y exhiben estructuras globales plausibles, especialmente en categorías como *ship*, *automobile*, *airplane* y *horse*, donde el aprendizaje del proceso inverso de difusión parece ser más sencillo. El análisis por épocas de AdaLN-Zero revela además una transición clara desde patrones ruidosos y con huecos hacia formas más densas y definidas conforme avanza el entrenamiento, ilustrando de manera didáctica la dinámica de aprendizaje de un modelo de difusión latente basado en Transformers.

En conjunto, los experimentos confirman de manera cualitativa las conclusiones del trabajo original: el uso de Transformers como *backbone* de modelos de difusión en el espacio latente es una alternativa viable a los U-Nets convolucionales, y la variante AdaLN-Zero proporciona un esquema de acondicionamiento especialmente efectivo cuando se dispone de una configuración de entrenamiento suficientemente estable. Además, el código desarrollado aporta una implementación clara y extensible que puede servir como punto de partida para futuros estudios sobre arquitecturas de difusión basadas en Transformers en distintos dominios y resoluciones.

En resumen, la re-implementación de DiT presentada aquí constituye un primer paso hacia el estudio sistemático de modelos de difusión basados en Transformers en diferentes escalas y dominios, ofreciendo una base de código sobre la cual explorar nuevas variantes arquitectónicas, *schedulers*, esquemas de condicionamiento y aplicaciones.

6. Trabajo futuro

Como trabajo futuro, se propone evaluar el uso de arquitecturas diferentes que podrían ser más adecuadas para dimensiones pequeñas (pues el espacio latente de $8 \times 8 \times 4$ utilizado en este trabajo produce una cantidad subóptima de tokens), ya sea utilizando parches de tamaño 1×1 o, inclusive, *MLPs*. Otro trabajo interesante sería incluir modelos de *embeddings* para producir condiciones más complejas, por ejemplo, el token de clase ([CLS]) de DINOv2 [3].

Referencias

- [1] Alexander Kolesnikov, Alexey Dosovitskiy, Dirk Weissenborn, Georg Heigold, Jakob Uszkoreit, Lucas Beyer, Matthias Minderer, Mostafa Dehghani, Neil Houlsby, Sylvain Gelly, Thomas Unterthiner, and Xiaohua Zhai. An image is worth 16x16 words: Transformers for image recognition at scale. 2021.
- [2] William Peebles and Saining Xie. Scalable diffusion models with transformers, 2023.
- [3] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shang-Wen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2023.